



Guru Chain Requirements for L2

- Changes Required to Host an OP Stack L2 -

2026.09

Gurufin Research Center

조광현

Tillo Tulkinbekov

Table of Contents

Overview	2
Document information	2
Summary of required changes	2
Guru Chain MUST DO Changes.....	3
1. eth_getProof error on "latest" (bug fix)	3
Change needed.....	5
2. receiptsRoot calculation for non-empty blocks (bug fix)	6
Changes needed.....	9
3. eth_getBlockReceipts on eth_call on DisputeGameFactory hits panic (bug fix).....	12
Change needed.....	12
4. failed to load L1 header: not found (feature).....	13
Error overview.....	13
Solution.....	14

Overview

This document contains the full report of the required and optional changes that have to be made on Guru Chain (L1) to make it capable of hosting OP Stack L2 chains.

Guru Chain is a Cosmos SDK chain running the cosmos/evm module. The OP Stack components (op-node, op-batcher, op-proposer, op-challenger and the Bedrock L1 contracts) are written against a go-ethereum compatible JSON-RPC surface and against Ethereum EVM semantics. The items collected here are exactly the points where the current Guru implementation diverges from that assumption and therefore breaks an L2 deployment.

Document information

Item	Value
Title	Guru Chain Requirements for L2
Category	Research
Author	Tillo Tulkinbekov
Created	August 6, 2026
Last updated	September 15, 2026
Target	Guru Chain (L1), cosmos/evm based
Consumer	OP Stack (Bedrock) L2

Summary of required changes

All four items below are mandatory. Items 1 to 3 are defects in the JSON-RPC layer, item 4 is a missing EVM feature that has to be added to the state machine.

No	Change	Type	Affected source
1	eth_getProof fails on the "latest" block tag	Bug fix	rpc/backend/account_info.go
2	receiptsRoot is always the empty root hash	Bug fix	rpc/backend/blocks.go, rpc/types/utils.go
3	Node panics on a query with an unknown block hash	Bug fix	rpc/backend/blocks.go
4	BLOCKHASH history is not kept by the EVM	Feature	x/vm/types, x/vm/keeper

Guru Chain MUST DO Changes

💡 These changes are **MUST DO** changes on the Guru Chain source code for supporting an OP Stack L2.

1. eth_getProof error on "latest" (bug fix)

OP Stack issues eth_getProof with the latest block tag. Guru translates the block tags into negative block numbers:

```
// file: rpc/types/block.go

const (
    EthPendingBlockNumber = BlockNumber(-2)
    EthLatestBlockNumber  = BlockNumber(-1) // latest = -1
    EthEarliestBlockNumber = BlockNumber(0)
)
```

eth_getProof calls this method internally:

```
// file: rpc/backend/account_info.go

// GetProof returns an account object with proof and any storage proofs
func (b *Backend) GetProof(address common.Address, storageKeys []string,
    blockNrOrHash rpctypes.BlockNumberOrHash) (*rpctypes.AccountResult, error) {
    blockNum, err := b.BlockNumberFromTendermint(blockNrOrHash)
    if err != nil {
        return nil, err
    }

    height := int64(blockNum)

    _, err = b.TendermintBlockByNumber(blockNum)
    if err != nil {
        // the error message imitates geth behavior
        return nil, errors.New("header not found")
    }

    // HERE
    // height == 0 => the condition only handles the 0 case.
    // -1 and -2 cases are always returned by
    // "cannot query with height < 0"
```

```

    // if the height is equal to zero, meaning the query condition of the block is
    either "pending" or "latest"
    if height == 0 {
        bn, err := b.BlockNumber()
        if err != nil {
            return nil, err
        }

        if bn > math.MaxInt64 {
            return nil, fmt.Errorf("not able to query block number greater than
MaxInt64")
        }

        height = int64(bn) // #nosec G115 -- checked for int overflow already
    }

    ctx := rpctypes.ContextWithHeight(height)
    clientCtx := b.clientCtx.WithHeight(height)

    // query storage proofs
    storageProofs := make([]rpctypes.StorageResult, len(storageKeys))

    for i, key := range storageKeys {
        hexKey := common.HexToHash(key)
        valueBz, proof, err := b.queryClient.GetProof(clientCtx, evmtypes.StoreKey,
evmtypes.StateKey(address, hexKey.Bytes()))
        if err != nil {
            return nil, err
        }

        storageProofs[i] = rpctypes.StorageResult{
            Key:    key,
            Value: (*hexutil.Big)(new(big.Int).SetBytes(valueBz)),
            Proof:    GetHexProofs(proof),
        }
    }

    // query EVM account
    req := &evmtypes.QueryAccountRequest{
        Address: address.String(),
    }

    res, err := b.queryClient.Account(ctx, req)
    if err != nil {

```

```

        return nil, err
    }

    // query account proofs
    accountKey := bytes.HexBytes(append(authtypes.AddressStoreKeyPrefix,
address.Bytes()...))
    _, proof, err := b.queryClient.GetProof(clientCtx, authtypes.StoreKey,
accountKey)
    if err != nil {
        return nil, err
    }

    balance, ok := sdkmath.NewIntFromString(res.Balance)
    if !ok {
        return nil, errors.New("invalid balance")
    }

    return &rpctypes.AccountResult{
        Address:      address,
        AccountProof: GetHexProofs(proof),
        Balance:      (*hexutil.Big)(balance.BigInt()),
        CodeHash:     common.HexToHash(res.CodeHash),
        Nonce:        hexutil.Uint64(res.Nonce),
        StorageHash:  common.Hash{}, // NOTE: Cosmos EVM doesn't have a storage hash.
        // TODO: implement?
        StorageProof: storageProofs,
    }, nil
}

```

Change needed

The guard only handles the 0 case, so the -1 (latest) and -2 (pending) cases keep the negative height and always fail with "cannot query with height < 0". Widening the comparison is enough:

```

// simple one line update

`if height == 0` => changed to => `if height <= 0`
// or if height == 0 || if height == -1 || if height == -2

// correlation:

// "earliest" (0)
// "latest" (-1)
// "pending" (-2)

```

```
// all can be resolved to "current block",
// so the change does not hurt the system
// after all it is just a query method
```

2. receiptsRoot calculation for non-empty blocks (bug fix)

The Guru (cosmos/evm) code has a bug in the calculation of the receiptsRoot for non-empty blocks. The block formatter hard-codes the empty root hash regardless of the receipts the block actually contains, which makes the L2 derivation reject the header.

```
// file: rpc/backend/blocks.go

// RPCBlockFromTendermintBlock returns a JSON-RPC compatible Ethereum block from a
// given Tendermint block and its block result.
func (b *Backend) RPCBlockFromTendermintBlock(
    resBlock *tmrpctypes.ResultBlock,
    blockRes *tmrpctypes.ResultBlockResults,
    fullTx bool,
) (map[string]interface{}, error) {
    ethRPCTxs := []interface{}{}
    block := resBlock.Block

    baseFee, err := b.BaseFee(blockRes)
    if err != nil {
        // handle the error for pruned node.
        b.logger.Error("failed to fetch Base Fee from pruned block. Check node
pruning configuration", "height", block.Height, "error", err)
    }

    msgs := b.EthMsgsFromTendermintBlock(resBlock, blockRes)
    for txIndex, ethMsg := range msgs {
        if !fullTx {
            hash := common.HexToHash(ethMsg.Hash)
            ethRPCTxs = append(ethRPCTxs, hash)
            continue
        }

        tx := ethMsg.AsTransaction()
        height := uint64(block.Height) //#nosec G115 -- checked for int overflow
already
        index := uint64(txIndex) //#nosec G115 -- checked for int overflow
already
        rpcTx, err := rpctypes.NewRPCTransaction(
            tx,
```

```

        common.BytesToHash(block.Hash()),
        height,
        index,
        baseFee,
        b.chainID,
    )
    if err != nil {
        b.logger.Debug("NewTransactionFromData for receipt failed", "hash",
tx.Hash().Hex(), "error", err.Error())
        continue
    }
    ethRPCTxs = append(ethRPCTxs, rpcTx)
}

bloom, err := b.BlockBloom(blockRes)
if err != nil {
    b.logger.Debug("failed to query BlockBloom", "height", block.Height, "error",
err.Error())
}

req := &evmtypes.QueryValidatorAccountRequest{
    ConsAddress: sdk.ConsAddress(block.Header.ProposerAddress).String(),
}

var validatorAccAddr sdk.AccAddress

ctx := rpctypes.ContextWithHeight(block.Height)
res, err := b.queryClient.ValidatorAccount(ctx, req)
if err != nil {
    b.logger.Debug(
        "failed to query validator operator address",
        "height", block.Height,
        "cons-address", req.ConsAddress,
        "error", err.Error(),
    )
    // use zero address as the validator operator address
    validatorAccAddr = sdk.AccAddress(common.Address{}.Bytes())
} else {
    validatorAccAddr, err = sdk.AccAddressFromBech32(res.AccountAddress)
    if err != nil {
        return nil, err
    }
}
}

```

```

    validatorAddr := common.BytesToAddress(validatorAccAddr)

    gasLimit, err := rpctypes.BlockMaxGasFromConsensusParams(ctx, b.clientCtx,
block.Height)
    if err != nil {
        b.logger.Error("failed to query consensus params", "error", err.Error())
    }

    gasUsed := uint64(0)

    for _, txsResult := range blockRes.TxsResults {
        // workaround for cosmos-sdk bug. https://github.com/cosmos/cosmos-
sdk/issues/10832
        if ShouldIgnoreGasUsed(txsResult) {
            // block gas limit has exceeded, other txs must have failed with same
reason.
            break
        }
        gasUsed += uint64(txsResult.GetGasUsed()) // #nosec G115 -- checked for int
overflow already
    }

    formattedBlock := rpctypes.FormatBlock(
        block.Header, block.Size(),
        gasLimit, new(big.Int).SetUint64(gasUsed),
        ethRPCTxs, bloom, validatorAddr, baseFee,
    )
    return formattedBlock, nil
}

```

The block formatter it calls:

```

// file: rpc/types/utils.go

// FormatBlock creates an ethereum block from a tendermint header and ethereum-
formatted
// transactions.
func FormatBlock(
    header cmmtypes.Header, size int, gasLimit int64,
    gasUsed *big.Int, transactions []interface{}, bloom ethtypes.Bloom,
    validatorAddr common.Address, baseFee *big.Int,
) map[string]interface{} {
    var transactionsRoot common.Hash
    if len(transactions) == 0 {
        transactionsRoot = ethtypes.EmptyRootHash
    }
}

```

```

    } else {
        transactionsRoot = common.BytesToHash(header.DataHash)
    }

    result := map[string]interface{}{
        "number":          hexutil.Uint64(header.Height), //nolint:gosec // G115 //
won't exceed uint64
        "hash":            hexutil.Bytes(header.Hash()),
        "parentHash":      common.BytesToHash(header.LastBlockID.Hash.Bytes()),
        "nonce":           ethtypes.BlockNonce{}, // PoW specific
        "sha3Uncles":      ethtypes.EmptyUncleHash, // No uncles in Tendermint
        "logsBloom":       bloom,
        "stateRoot":       hexutil.Bytes(header.AppHash),
        "miner":           validatorAddr,
        "mixHash":         common.Hash{},
        "difficulty":      (*hexutil.Big)(big.NewInt(0)),
        "extraData":       "0x",
        "size":            hexutil.Uint64(size), //nolint:gosec // G115 // size
won't exceed uint64
        "gasLimit":        hexutil.Uint64(gasLimit), //nolint:gosec // G115 // gas
limit won't exceed uint64
        "gasUsed":         (*hexutil.Big)(gasUsed),
        "timestamp":       hexutil.Uint64(header.Time.Unix()), //nolint:gosec //
G115 // won't exceed uint64
        "transactionsRoot": transactionsRoot,
        "receiptsRoot":    ethtypes.EmptyRootHash,

        "uncles":          []common.Hash{},
        "transactions":    transactions,
        "totalDifficulty": (*hexutil.Big)(big.NewInt(0)),
    }

    if baseFee != nil {
        result["baseFeePerGas"] = (*hexutil.Big)(baseFee)
    }

    return result
}

```

Changes needed

The receipts root has to be derived from the receipts of the block. The formatted block is post-processed with a helper that rebuilds the receipt list and derives the trie root, falling back to the empty root hash whenever a receipt cannot be reconstructed:

```

// file: rpc/backend/blocks.go

// RPCBlockFromTendermintBlock returns a JSON-RPC compatible Ethereum block from a
// given Tendermint block and its block result.
func (b *Backend) RPCBlockFromTendermintBlock(
    resBlock *tmrpctypes.ResultBlock,
    blockRes *tmrpctypes.ResultBlockResults,
    fullTx bool,
) (map[string]interface{}, error) {

    ...

    formattedBlock := rpctypes.FormatBlock(
        block.Header, block.Size(),
        gasLimit, new(big.Int).SetUint64(gasUsed),
        ethRPCTxs, bloom, validatorAddr, baseFee,
    )

    // one-liner new addition
    // recalculates and overrides the empty root hash by actual data from block
    formattedBlock["receiptsRoot"] = b.blockReceiptsRoot(resBlock, blockRes)

    return formattedBlock, nil
}

// helper function
func (b *Backend) blockReceiptsRoot(resBlock *tmrpctypes.ResultBlock, blockRes
*tmrpctypes.ResultBlockResults) common.Hash {
    msgs := b.EthMsgsFromTendermintBlock(resBlock, blockRes)
    if len(msgs) == 0 {
        return ethtypes.EmptyRootHash
    }

    receipts := make([]*ethtypes.Receipt, len(msgs))
    for _, ethMsg := range msgs {
        txResult, err := b.GetTxByEthHash(common.HexToHash(ethMsg.Hash))
        if err != nil {
            b.logger.Debug("blockReceiptsRoot: tx not found", "hash", ethMsg.Hash,
"error", err.Error())
            return ethtypes.EmptyRootHash
        }

        ethTxIndex := int(txResult.EthTxIndex)
        if ethTxIndex < 0 || ethTxIndex >= len(receipts) {

```

```

        b.logger.Debug("blockReceiptsRoot: unexpected eth tx index", "hash",
ethMsg.Hash, "index", txResult.EthTxIndex)
        return ethtypes.EmptyRootHash
    }

    // Cumulative gas used, computed identically to formatTxReceipt.
    cumulativeGasUsed := uint64(0)
    for _, res := range blockRes.TxsResults[0:txResult.TxIndex] {
        cumulativeGasUsed += uint64(res.GasUsed) // #nosec G115 -- checked for
int overflow already
    }
    cumulativeGasUsed += txResult.CumulativeGasUsed

    status := uint64(ethtypes.ReceiptStatusSuccessful)
    if txResult.Failed {
        status = ethtypes.ReceiptStatusFailed
    }

    logs, err := TxLogsFromEvents(blockRes.TxsResults[txResult.TxIndex].Events,
int(txResult.MsgIndex))
    if err != nil {
        b.logger.Debug("blockReceiptsRoot: failed to parse logs", "hash",
ethMsg.Hash, "error", err.Error())
    }

    receipt := &ethtypes.Receipt{
        Type:          ethMsg.AsTransaction().Type(),
        Status:         status,
        CumulativeGasUsed: cumulativeGasUsed,
        Logs:          logs,
    }
    receipt.Bloom = ethtypes.CreateBloom(receipt)
    receipts[ethTxIndex] = receipt
}

for _, receipt := range receipts {
    if receipt == nil {
        b.logger.Debug("blockReceiptsRoot: missing receipt for eth tx index",
"height", resBlock.Block.Height)
        return ethtypes.EmptyRootHash
    }
}

return ethtypes.DeriveSha(ethtypes.Receipts(receipts), trie.NewStackTrie(nil))

```

```
}
```

3. eth_getBlockReceipts on eth_call on DisputeGameFactory hits panic (bug fix)

There is an RPC path that crashes the node on nil headers. CometBFT returns a non-nil ResultHeader with a nil Header for an unknown hash, so the existing nil check is bypassed and the dereference panics:

```
func (b *Backend) BlockNumberFromTendermintByHash(blockHash common.Hash) (*big.Int, error) {
    resBlock, err := b.rpcClient.HeaderByHash(b.ctx, blockHash.Bytes())
    if err != nil {
        return nil, err
    }

    // PROBLEM is HERE
    // CometBFT returns the non-nil ResultHeader with a nil Header
    // for unknown hash
    // bypassing this nil check, the nil hash causes node crash with panic

    if resBlock == nil {
        return nil, errors.Errorf("block not found for hash %s", blockHash.Hex())
    }

    // PANIC HERE
    // if the Header os nil => panic!

    return big.NewInt(resBlock.Header.Height), nil
}
```

Change needed

Extend the guard to the inner Header field. The change is harmless for every other caller, the method simply reports "block not found" instead of crashing the node:

```
func (b *Backend) BlockNumberFromTendermintByHash(blockHash common.Hash) (*big.Int, error) {
    resBlock, err := b.rpcClient.HeaderByHash(b.ctx, blockHash.Bytes())
    if err != nil {
        return nil, err
    }

    // protect against the panic
```

```

// make sure Header is not nil
// Harmless change
if resBlock == nil || resBlock.Header == nil {
    return nil, errors.Errorf("block not found for hash %s", blockHash.Hex())
}

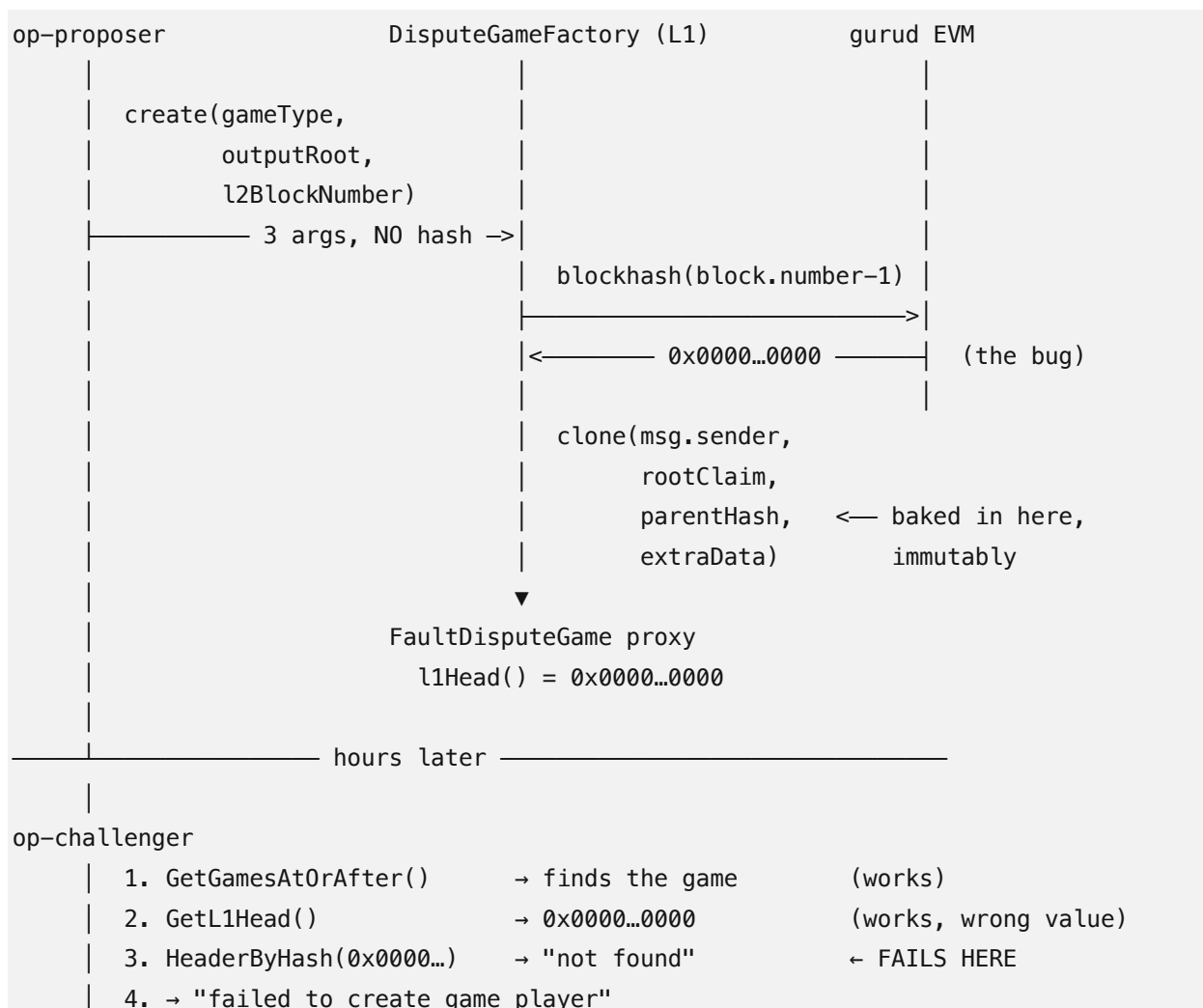
return big.NewInt(resBlock.Header.Height), nil
}

```

4. failed to load L1 header: not found (feature)

Error overview

The proposer and challenger interaction looks as follows:



DisputeGameFactory.sol:156-170

...

```

bytes32 parentHash = blockhash(block.number - 1);
...
proxy_ = IDisputeGame(address(impl).clone(
    abi.encodePacked(msg.sender, _rootClaim, parentHash, _extraData)));
// CWIA layout: [0,20) creator | [20,52) rootClaim | [52,84) parentHash | [84,...)
extraData

```

- BLOCKHASH is requested for the history (n-1) from the Guru EVM.
- The EVM does not store the block hash history, so it returns 0x0000...0000.

Description

- When the proposer creates a dispute game on the L1 smart contract, the default 0x0000...0000 block hash is stored as the parentHash.
- There is no error in the proposer lifecycle, because the parentHash is never re-used by the proposer itself.
- When the challenger enters the game and replays it, it fetches the corresponding block hash from Guru Chain and compares it with the value stored in the L1 smart contract, and the lookup fails.

Solution

There is only one correct way to fix this: store the block hash history in the EVM. Ethereum keeps the history of the last 256 blocks, so the same behaviour is expected by the contracts.

x/vm/types/key.go

```

// file: x/vm/types/key.go

const (
    prefixCode = iota + 1
    prefixStorage
    prefixParams
    prefixCodeHash
)

var (
    KeyPrefixCode      = []byte{prefixCode}
    KeyPrefixStorage   = []byte{prefixStorage}
    KeyPrefixParams    = []byte{prefixParams}
    KeyPrefixCodeHash = []byte{prefixCodeHash}
)

```

Changes:

```
// BlockHashWindow is how many recent block hashes are retained for the
// BLOCKHASH opcode. Last 256 saved like in Ethereum
const BlockHashWindow = 256

func BlockHashKey(height int64) []byte // KeyPrefixBlockHash
```

x/vm/keeper/block_hash.go (new file)

A new keeper file holds the setter and the getter for the retained hashes. Pruning a single height per block keeps the store bounded without an unbounded sweep:

```
// file: x/vm/keeper/block_hash.go (new file)

// SetBlockHash records the CometBFT header hash for the given height
func (k Keeper) SetBlockHash(ctx sdk.Context, height int64, hash []byte) {
    if len(hash) == 0 {
        return
    }
    store := ctx.KVStore(k.storeKey)
    store.Set(types.BlockHashKey(height), hash)

    // Prune the entry that just left the window. Pruning a single height per
    // block keeps the store bounded without an unbounded sweep.
    if prune := height - types.BlockHashWindow; prune >= 0 {
        store.Delete(types.BlockHashKey(prune))
    }
}

// GetBlockHash returns the CometBFT header hash recorded for height
func (k Keeper) GetBlockHash(ctx sdk.Context, height int64) (common.Hash, bool) {
    bz := ctx.KVStore(k.storeKey).Get(types.BlockHashKey(height))
    if len(bz) == 0 {
        return common.Hash{}, false
    }
    return common.BytesToHash(bz), true
}
```

x/vm/keeper/abci.go

```
// file: x/vm/keeper/abci.go

func (k *Keeper) BeginBlock(ctx sdk.Context) error {
    logger := ctx.Logger().With("begin_block", "evm")
```

```

    res, err := k.BaseFee(ctx, &evmtypes.QueryBaseFeeRequest{})
    ...
}

```

Changes:

```

func (k *Keeper) BeginBlock(ctx sdk.Context) error {
    logger := ctx.Logger().With("begin_block", "evm")

    // Record this block's CometBFT hash.
    k.SetBlockHash(ctx, ctx.BlockHeight(), ctx.HeaderHash())

    res, err := k.BaseFee(ctx, &evmtypes.QueryBaseFeeRequest{})
    ...
}

```

x/vm/keeper/state_transition.go

```

// file: x/vm/keeper/state_transition.go

func (k Keeper) GetHashFn(ctx sdk.Context) vm.GetHashFunc {
    ...

    case ctx.BlockHeight() > h:
        histInfo, err := k.stakingKeeper.GetHistoricalInfo(ctx, h)
        if err != nil {
            k.Logger(ctx).Debug("error while getting historical info", ...)
            return common.Hash{}
        }
        header, err := cmttypes.HeaderFromProto(&histInfo.Header)
        ...
        return common.BytesToHash(header.Hash()) // <- nil -> zero hash
    ...
}

```

Changes:

```

func (k Keeper) GetHashFn(ctx sdk.Context) vm.GetHashFunc {
    ...
}

```

```

case ctx.BlockHeight() > h:

    // Preferred source: the hash CometBFT gave us for that height
    if hash, found := k.GetBlockHash(ctx, h); found {
        return hash
    }

    // Fallback to original version
    histInfo, err := k.stakingKeeper.GetHistoricalInfo(ctx, h)
    if err != nil {
        k.Logger(ctx).Debug("error while getting historical info", ...)
        return common.Hash{}
    }
    header, err := cmttypes.HeaderFromProto(&histInfo.Header)
    ...
    return common.BytesToHash(header.Hash()) // <- nil -> zero hash
...
}

```